

Measuring Development Time Efficiency and Execution Speed Between C and Python Through Controlled Tasks



David Sinambela - Fiko A. van Houten - Richtjhie H. Agusta | Supervised by Said Achmad & Maulin Nasari

BACKGROUND & MOTIVATION

the core trade-off
C compiles directly to machine code — fast runtime, low overhead. **Python runs via bytecode** interpreter — **slower execution, but drastically faster to write and prototype.**

the gap in literature
Prior work measures speed or productivity — never both simultaneously. **This study fills that gap using identical tasks, same conditions, and dual metrics for a fair side-by-side comparison.**

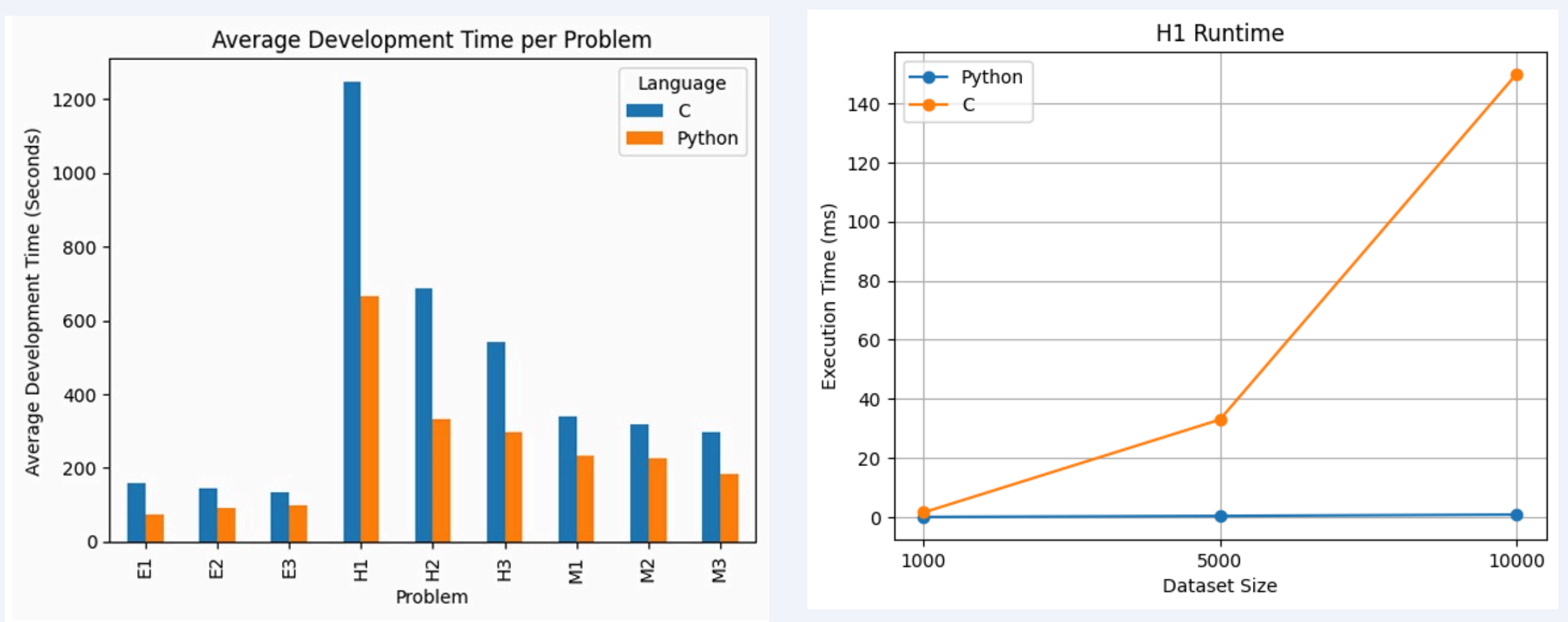
METHODOLOGY

controlled tasks
9 problems across 3 levels — Easy, Medium, Hard — solved in both C and Python by each volunteer under identical conditions.

volunteer programmers
Experienced in both languages. No AI allowed. Time tracked from blank file to passing test cases. All data anonymized.

4 metrics tracked
Development time (min) · Lines of Code · Execution time (ms) · Memory usage (MB) — all on a fixed hardware environment.

RESULTS



key patterns
Python is consistently faster to develop for Easy and Medium tasks — sometimes by over 2× (H1)

C was consistently slower to develop across all 9 tasks. The gap is most extreme at H1 — C took nearly 2× longer than Python.

CONCLUSION & FINDING

When put side by side on the same problems, under the same conditions — Python was 1.6× faster to develop (3.8 min vs 6.1 min avg) and used 56% fewer lines of code.. The more meaningful contribution of this study is not the result itself, but the **framework: measuring both dimensions simultaneously, with real volunteers, under controlled conditions.**

finding 1 - developer speed
Across easy and medium tasks, **Python required significantly less time** — aligning with broader literature on high-level language productivity advantages.

finding 2 - runtime efficiency
C produced more execution-efficient solutions when algorithm type was controlled — as seen in H2 (matrix transpose) where C was 31× faster at large scale

finding 3 - hard tasks
At hard difficulty, **C's explicit control structure may reduce developer ambiguity** — narrowing **Python's typical productivity lead.**

finding 4 - framework value
No prior study jointly measures development time and runtime efficiency on identical tasks with the same participants — this study proposes that framework.

LIMITATIONS & RECOMMENDATIONS FOR FUTURE RESEARCH

scope & problem design
Nine problems across three levels was ambitious for available resources. **Future work should narrow scope** — fewer, more precisely constrained problems yield cleaner data.

volunteer recruitment
Finding programmers fluent in both languages willing to complete all tasks proved difficult. Build recruitment into your timeline early; smaller, consistent pools outperform large but inconsistent ones.

hardware standardization
Environmental variables were partially unresolved at data collection. All conditions — hardware, OS, compiler versions — must be locked before the experiment begins, not during it.

[1]Ritonga, "Studi Literatur Perbandingan Bahasa Pemrograman C++ Dan Bahasa Pemrograman Python Pada Algoritma Pemrograman," Jurnal Teknik Informatika dan Teknologi Informatika, pp. 56-63, 2023. [2] Y. He, "A Comparative Study of the Significance of Different Programming Languages," Academic Journal of Computing & Information Science, pp. 45-50, 2025. [3] A. Kumar, "Performance comparison of instrument automation pipelines using different programming languages," Scientific Reports, pp. 1-14, 2023. [4] V. Stokel, "An Empirical Study on the Performance and Energy Usage of Compiled Python Code," Proceedings of the 29th International Conference on Evaluation and Assessment in Software Engineering, pp. 45-56, 2025. [5] P. Jain, "Comparative Review of Python and C: Choosing the Right Language for Beginners," International Journal of Engineering Trends and Applications, vol. 11, no. 3, pp. 148-156, 2024. [6] R. Kosma, Algoritma Dan Programming Konsep Dasar, Logika, dan Implementasi dengan C++ & Python, Pustaka: Sinar Media, 2025. [7] R. Kumar, "Python Versus C Language: A Comparison," International Journal of Trend in Research and Development, pp. 171-173, 2022. [8] A. Gajin, "C vs Python: A Cautious Look with Industry Opinion," International Journal for Research, pp. 55-64, 2023. [9] S. Ali, "A Pragmatic Comparison of Four Different Programming Languages," ScienceOpen Preprints, pp. 1-13, 2021. [10] A. Azevedo, "The Influence of Programming Languages on Computational Efficiency and Performance," International Journal of Research and Innovation in Social Science, pp. 285-288, 2025. [11] P. Nardack, "A Analysis of programming languages used in solving energy problems," Environmental Science, Computer Science, Engineering, pp. 1-6, 2022. [12] Y. Zhang, "Comparative study of the execution efficiency of Python and C++ based on topological sorting," International Conference on Machine Learning and Automation, pp. 13-17, 2024. [13] L. Miller, "Comparative Study of Python and C++ in High-Performance AI Model Deployment," International Journal of Computer Science, pp. 1-5, 2025. [14] H. Patel, "The Enduring Relevance of C Programming: Analysing Its Efficiency, Portability and Role in Modern Computing," A Global Journal of Interdisciplinary Studies, pp. 546-550, 2025. [15] M. J. I. Antonio, "A Discerning Comparison between C++ and Python in Game Development," pp. 1-14, 2024. [16] B. R. Maheshwari, "Comparative Analysis of Program Execution Time Required by Python, R and Julia Compiler," Indian Journal of Computer Science and Technology, pp. 25-27, 2024. [17] Y. Tao, "Beyond Functional Correctness: An Exploratory Study on the Time Efficiency of Programming Assignments," International Conference on Software Engineering: Software Engineering Education and Training, pp. 320-330, 2024. [18] R. Pereira, "Ranking programming languages by energy efficiency," Science of Computer Programming, vol. 205, no. 102609, pp. 1-30, 2021. [19] P. Rysak, "Comparative analysis of C and Python on the basis of the execution time of applications implementing selected algorithms," Journal Computer Science Institute, vol. 26, pp. 93-99, 2023. [20] H. Kishorbal, "THE ENDURING RELEVANCE OF C PROGRAMMING: ANALYSING ITS EFFICIENCY, PORTABILITY, AND ROLE IN MODERN COMPUTING," A Global Journal of Interdisciplinary Studies, pp. 546-550, 2025. [21] V. M. Ionescu, "Investigating the performance of MicroPython and C on ESP32 and STM32 microcontrollers," International Symposium for Design and Technology in Electronic Packaging, vol. 10, pp. 234-237, 2020. [22] A. Ujwalika, "A comparative study on usage of Python and C/C++ for firmware development," International Journal of Advance Research, Ideas and Innovation in Technology, vol. 6, no. 3, pp. 557-559, 2020. [23] V. Cattini, "Comparative Review of JAVA and Python," International Journal of Research and Development in Applied Science and Engineering, vol. 21, no. 1, pp. 1-6, 2021. [24] M. Marigaona, "Static Analysis for ADA, C/C++ and Python Different Languages, Different Needs," Joint publication with Ada User Journal, vol. 42, no. 3, pp. 7-40, 2021. [25] U. Schmitt, "symp2c: From symbolic expressions to fast C/C++ functions and ODE," Astronomy and Computing, vol. 42, pp. 1-11, 2023. [26] R. A. Ouyang, "Comparative Analysis on the Evaluation of the Complexity of C, C++, Java, PHP and Python Programming Languages based on Habited Softwars," International Journal of Computer and Information Technology, vol. 12, no. 1, pp. 8-19, 2023. [27] J. O. Ogala, "COMPARATIVE ANALYSIS OF C, C++, C# AND JAVA PROGRAMMING LANGUAGES," Global Scientific Journal, vol. 8, no. 5, pp. 1900-1913, 2020. [28] M. Shoaib, "A Comparative Study of Contemporary Programming Languages in Implementation of Classical Algorithms," Journal of Information & Communication Technology, vol. 14, no. 1, pp. 23-31, 2021. [29] A. Kaur, "A Comparative Study of Static Code Analysis tools for Vulnerability Detection in C/C++ and JAVA Source Code," ScienceDirect, pp. 1-7, 2023. [30] S. Sundaramoorthy, "A systematic analysis on performance and computational complexity of sorting algorithms," Discover Computing, pp. 1-30, 2025.